TicketPilot — HLD & LLD

Oct 21, 2026 · @Charudatt Palvi

Problem Statement

Food-delivery support today is handled one of two ways: a rule-based chatbot that matches keywords to templates and gets stuck the moment a customer phrases a problem differently than expected, or a human agent who has to manually look up the order, check the account, and decide what to do for every single ticket, including the simple ones. The first frustrates customers; the second doesn't scale.

Most rule-based systems also can't tell the difference between a minor delay and a genuinely lost order without a human checking the data themselves, because they never look at real data — they only match the words in the message to a pre-written rule.

TicketPilot is built to close this gap: an agent that actually investigates each ticket using real order and account data before deciding what to do, resolves the simple cases on its own, and escalates or requests approval for anything it shouldn't decide alone.

What is TicketPilot?

TicketPilot is an AI-powered support system for a food-delivery platform. When a customer writes in with a problem, instead of matching keywords to a canned reply, TicketPilot investigates the issue first — it looks up the real order, checks the real account history, searches the help-center knowledge base — and only then decides what to do: answer directly, escalate to a human, or propose an action (like a refund) that a human has to approve before it actually happens.

What "AI Agent" means here: an AI Agent is a piece of software built on a large language model (LLM) that can read a situation, decide on its own which tool to call to gather more information, and choose the next step — without someone telling it exactly what to do at each stage. In TicketPilot, the agent can read ticket data and knowledge-base articles on its own, but it can never execute an action that changes real data (a refund, a credit, closing a complaint) without a human approving it first.

At a glance: Customer sends a message → Agent investigates (checks order status, account history, help articles) → Agent decides: reply directly, escalate to a human, or propose an action → anything that changes real data waits in an approval queue until a

human signs off.

HLD

Milestone: TicketPilot v1.0.0

Client: QuickCrave

Date: 27th Oct 2026

Author: Harshal Pawar

The Client Need

QuickCrave, a food-delivery platform, wants to reduce support-ticket resolution time and cut down on tickets that unnecessarily reach human agents, without making customers feel like they're talking to a wall. They need a system that can resolve routine delivery issues (delays, missing items, minor complaints) on its own, while keeping a human in control of anything that costs the company money (refunds, credits) or needs judgment (abusive messages, policy exceptions, unclear complaints).

The Proposed Solution

TicketPilot sits between the customer and the support team. When a ticket comes in, the agent reads it, calls tools to check the real order and account data, and reasons about what kind of situation this is. Simple, well-understood cases (a delayed order, a question answerable from the help center) are resolved immediately with a specific, data-backed reply. Cases that need a refund or account credit are prepared as a proposed action and placed in a queue for a support manager to approve or reject — TicketPilot never issues a refund or credit on its own. Cases it can't confidently resolve (ambiguous complaints, abusive messages, anything outside its scope) are escalated directly to a human with a summary of what the agent found.

Solution Flow Diagram

See TicketPilot_2_Investigation_Flow_Sequence.drawio (open in diagrams.net) — referenced in full under Diagrams below.

Step-by-Step Flow Description

Step 1: Customer Submits a Ticket

Customer submits a ticket through the app — a free-text message plus an order reference, if any.

Step 2: Agent Decides Which Tools to Call

TicketPilot's agent reads the ticket and decides which tools it needs: check_order_status, check_account_history, search_knowledge_base.

Step 3: Agent Investigates

The agent calls those tools and gets back real data — order status, past ticket history, relevant help articles.

Step 4: Agent Decides the Next Step

Based on what it finds, the agent takes one of three paths: **Resolve directly** (reply sent immediately, ticket marked resolved), **Propose an action** (for a refund/credit, calls propose_action, which writes a pending row to the approvals queue — nothing happens yet), or **Escalate** (ticket handed to a human support agent with the investigation summary attached).

Step 5: Support Manager Reviews a Proposed Action

For a proposed action, a support manager reviews it in the approval queue and approves or rejects it.

Step 6: Approval Executed

On approval, the refund/credit is actually issued, the ticket is marked resolved, and an audit log entry is written.

Step 7: Rejection Routed to a Human

On rejection, the ticket is routed to a human for manual handling, with the rejection reason logged.

Step 8: Customer Reopens If Unsatisfied

If the customer isn't satisfied with a resolved ticket, they can reopen it from the same ticket thread. This appends to the ticket's existing message history rather than creating a brand-new ticket, and moves its status back to investigating so the agent re-engages with full context of what was already discussed.

LLD

Milestone: TicketPilot v1.0.0

Client: QuickCrave

Date: 27th Oct 2026

Author: Harshal Pawar

1. Package Details

Package Name: ticketpilot_das

Purpose: Three modules inside the ticketpilot_das package:

- ticket_management — handles ticket submission, reopening, the agent investigation loop, and reading ticket/order/account data.
- approval_engine — handles the approval queue: listing pending proposed actions, approving, and rejecting them.
- auth — handles login and JWT issuance for the support manager role; no separate auth microservice, this lives inside the same package.

ticket_management and approval_engine meet at the Approval table: ticket_management's agent writes a pending row to it (via propose_action), and approval_engine both reads and writes it (on approve/reject), then calls back into ticket_management to update the ticket's own status. Both modules call into auth's db_operations to validate a JWT and resolve the requesting user, but auth itself never reads or writes ticket/approval data.

2. Module Responsibility

ticket_management

- Expose the endpoints for submitting a ticket, reopening one, fetching one ticket, and listing tickets.

- Own the AgentOrchestrator — the tool-calling loop described in full under section 3 below.
- Read Order and Customer tables directly for its own lookup tools; call Qdrant for search_knowledge_base.
- When the agent's decision is propose_action, call into approval_engine's db_operations to create the pending approval row — this is the one place the two modules meet.
- Write to the Ticket table for the ticket's own state (investigating/resolved/escalated/pending_approval), and to the new TicketMessage table for every message in the ticket's conversation — the customer's original message, each tool call result, the agent's reply, and any reopen message. TicketMessage is what gives a ticket memory across multiple exchanges, and what reopening appends to instead of starting a new ticket.

approval_engine

- Expose the endpoints for listing pending approvals, approving an action, and rejecting one.
- Own the Approval and AuditLog tables.
- On approve: execute the refund/credit (simulated for the MVP), call back into ticket_management's db_operations to mark the ticket resolved, write an AuditLog row.
- On reject: call back into ticket_management's db_operations to mark the ticket escalated with the rejection reason, write an AuditLog row.
- Neither module owns customer/order data beyond what it reads.

auth

- Expose the login endpoint: validates email/password against the User table, returns a JWT.
- Own the User table — the only table it owns; it has no ticket/approval data of its own.
- Every other module's routes validate the caller's JWT via a shared dependency that calls into auth's db_operations, rather than each module re-implementing token validation.

3. AgentOrchestrator — Tool-Calling Loop, Context, and Governance

How the loop actually works

AgentOrchestrator sends the LLM the ticket's full message transcript (see Conversation Context below) plus the exact schema of every tool it's allowed to call — name, parameters, and a description — for check_order_status, check_account_history, search_knowledge_base, propose_action, and escalate_ticket. The LLM does not execute anything itself: it responds with which tool to call and what arguments to use (for example, check_order_status(order_id=4521)). AgentOrchestrator then actually runs that Python function against the real database/Qdrant, and sends the real result back to the LLM as the next message in the same conversation. The LLM reads that result and decides whether it has enough information to finish, or whether it needs to call another tool next — this can repeat several times (for example: check the order first, then search the knowledge base for the relevant policy) before the LLM either replies directly in plain text (the ticket is then resolved) or calls one of the two decision tools — propose_action or escalate_ticket — instead of another investigation tool call. Calling either decision tool IS the decision: each one updates the ticket's own status as a side effect, so there is no separate decision step afterwards.

Conversation Context

A ticket is not always one message and one reply — a customer may send a follow-up, or reopen a resolved ticket. Every message in a ticket's lifetime (the customer's original message, each tool call and its result, the agent's reply, any follow-up or reopen message) is appended, in order, to the TicketMessage table. On any new message for an existing ticket, AgentOrchestrator loads that full transcript and sends it back to the LLM as context before adding the new message — so the agent never starts from zero and never repeats investigation it already did. This is standard multi-turn chat context, scoped to one ticket; no separate memory system is built for it.

Tool Governance

The LLM's tool-calling API is a closed set by design: it can only choose among the exact tools registered with it (check_order_status, check_account_history, search_knowledge_base, propose_action, escalate_ticket) and can never invent or call anything outside that list. On top of that, each tool function enforces its own scope in code regardless of what the LLM asks for: check_order_status only ever reads orders tied to the ticket's own customer_id, propose_action never executes anything — it only ever writes a pending row to Approval — and escalate_ticket only ever changes the ticket's

own status and hands it to a human; neither has any access to money movement or another customer's data. There is no code path available to the agent that can directly issue a refund or change account data; that only ever happens through approveAction, which requires a human. This is the same principle as a human support agent only having access to the internal systems their company gives them — a limited toolbox restricts what actions are possible, it does not restrict how the agent understands the customer's message, which is what separates this from keyword-matching systems.

4. Endpoints

4.1 ticket_management endpoints

POST /ticketmanagement/submitTicket

Function Name: submit_ticket

Description: Accepts a new customer ticket and runs the agent investigation loop synchronously — the request stays open until the agent has a final reply, escalation, or proposed action; the frontend simply waits (typically a few seconds).

Inputs: customer_id, order_id (required — the agent cannot investigate without a real order to check), message (free text)

Outputs: ticket_id, status (resolved / escalated / pending_approval), messages (the full conversation so far, including the agent's reply if resolved)

Happy Flow: Ticket is saved → first message written to TicketMessage → AgentOrchestrator runs → tools are called as needed → agent decides reply/escalate/propose_action → ticket status updated, reply written to TicketMessage → response returned.

Error Flow: Invalid customer_id or order_id → 400 Bad Request. Missing order_id → 400 Bad Request. LLM/tool-calling failure → ticket is marked escalated with a system note, so no ticket is ever silently lost.

POST /ticketmanagement/reopenTicket

Function Name: reopen_ticket

Description: Lets the customer reopen a resolved ticket they're not satisfied with, instead of filing a new one.

Inputs: ticket_id, message (free text — why they're reopening it)

Outputs: ticket_id, status (investigating)

Happy Flow: Ticket's current status validated as resolved → new message appended to TicketMessage → ticket status set to investigating → AgentOrchestrator runs again with the full prior transcript as context.

Error Flow: Ticket not found → 404 Not Found. Ticket isn't currently resolved (still investigating, or already escalated) → 409 Conflict.

GET /ticketmanagement/getTicket

Function Name: get_ticket

Description: Returns one ticket's full detail, including its message history and investigation trail.

Inputs: ticket_id

Outputs: 200 OK — the ticket record (ticket_id, customer_id, order_id, status, created_date, resolved_date) plus its ordered messages list (sender, message, created_date). The raw tool-call trail is kept internally for audit purposes but is not returned to the frontend — customers only ever see the human-readable conversation.

Happy Flow: Ticket looked up by ticket_id → its TicketMessage rows loaded in order → returned together.

Error Flow: Ticket not found → 404 Not Found.

GET /ticketmanagement/getTickets

Function Name: get_tickets

Description: Lists tickets for the inbox view, filterable by status.

Inputs: optional status, page, pageSize (defaults applied if omitted).

Outputs: 200 OK — one page of ticket rows plus totalCount.

Happy Flow: Applies the status filter directly in the query, paginates at the database level, returns the page plus totalCount.

Error Flow: Database/internal failure → 500 Internal Server Error.

4.2 auth endpoints

POST /auth/login

Function Name: login

Description: Validates a support manager's credentials and issues a JWT used by every other endpoint.

Inputs: email, password

Outputs: 200 OK — access_token, token_type, user_id, role

Happy Flow: User looked up by email → password checked against the stored hash → JWT issued.

Error Flow: Email not found, or password incorrect → 401 Unauthorized.

4.3 approval_engine endpoints

GET /approvalengine/getPendingApprovals

Function Name: get_pending_approvals

Description: Lists proposed actions awaiting a support manager's decision.

Inputs: optional page, pageSize.

Outputs: 200 OK — the list of pending approvals (approval_id, ticket_id, action_type, amount, reason, status).

Happy Flow: Reads Approval where status = pending, paginates, returns the page plus totalCount.

Error Flow: Database/internal failure → 500 Internal Server Error.

POST /approvalengine/approveAction

Function Name: approve_action

Description: Executes a previously proposed action (refund or credit) after a support manager approves it.

Inputs: approval_id (path); the approver is identified from the caller's JWT, not a request field

Outputs: 200 OK — the updated approval (approval_id, ticket_id, action_type, amount, reason, status: approved, approver_id, decided_date)

Happy Flow: Approval row validated as pending → refund/credit executed against the order → ticket marked resolved → audit log entry written.

Error Flow: Approval already actioned → 409 Conflict. Refund execution fails → approval stays pending, error surfaced to the approver.

POST /approvalengine/rejectAction

Function Name: reject_action

Description: Rejects a proposed action; the ticket is routed to a human for manual handling instead.

Inputs: approval_id (path), reason (optional); the approver is identified from the caller's JWT, not a request field

Outputs: 200 OK — the updated approval (approval_id, ticket_id, action_type, amount, reason, status: rejected, approver_id, decided_date)

Happy Flow: Approval row marked rejected → ticket status set to escalated with the rejection reason attached → audit log entry written.

Error Flow: Approval already actioned → 409 Conflict.

5. Database Schema

The full database schema is maintained in the attached TicketPilot_DB_Design.xlsx workbook.

6. Package Structure

Package Name: ticketpilot_das

```
ticketpilot_das/                  # repo root
  setup.py                        # declares external PyPI dependencies
  config.ini.example              # copy to config.ini and fill in
  ticketpilot_das/                # the actual Python package
    main.py                       # FastAPI app entry point, registers all
three routers
    utils/
      ConfigReader.py            # thin wrapper around the standard
configparser module
      Logger.py                  # thin wrapper around the standard logging
module
      CommonEnums.py             # TicketStatus, ApprovalStatus,
ApprovalActionType, UserRole, OrderStatus
      Database.py                # shared SQLAlchemy engine/session/Base
    ticket_management/
      entity/DbModels.py          # Customer, Order, Ticket, TicketMessage
(SQLAlchemy)
      entity/PydanticModels.py    # request/response shapes for
```

```
submit/reopen/get/list ticket
      db_operations/Ticket.py     # submit_ticket, reopen_ticket, get_ticket,
get_tickets
      routes/Ticket.py            # the 4 endpoints
      AgentOrchestrator.py        # the tool-calling loop
      tools/
        OrderLookupTool.py        # check_order_status
        AccountLookupTool.py      # check_account_history
        KnowledgeSearchTool.py    # search_knowledge_base (Qdrant)
        ApprovalTool.py           # propose_action
        EscalateTool.py           # escalate_ticket
    approval_engine/
      entity/DbModels.py          # Approval, AuditLog (SQLAlchemy)
      entity/PydanticModels.py    # request/response shapes for the 3
endpoints
      db_operations/Approval.py   # create_pending_approval, approve_action,
reject_action
      routes/Approval.py          # the 3 endpoints
    auth/
      entity/DbModels.py          # User (SQLAlchemy)
      entity/PydanticModels.py    # request/response shapes for login
      db_operations/Auth.py       # credential check, JWT issuance/validation
(shared get_current_user dependency)
      routes/Auth.py              # the 1 endpoint
```

7. Open Questions

- Exact LLM provider, model, and embedding model to pin in setup.py — confirm before build.
- Whether a reopened ticket needs a cap on how many times it can be reopened, or stays unlimited for the MVP.
- Whether auth needs more than one role (e.g. a separate admin role) for the MVP, or support_manager alone is enough.
- Whether approveAction/rejectAction need a stricter per-approval-amount role check (e.g. only an admin can approve above a threshold), or any authenticated support manager can act on any amount — the MVP assumes the latter.
- Refund execution is simulated for the MVP (no real payment-gateway call) — confirm this is acceptable for the resume-facing demo.

Diagrams

Four `.drawio` files accompany this document (open in diagrams.net):

File	Shows
TicketPilot_1_System_Architecture.drawio	Frontend, backend, database, Qdrant, and the LLM provider, and how they connect
TicketPilot_2_Investigation_Flow_Sequence.drawio	Step-by-step sequence of a ticket being investigated, resolved/escalated, and (for a proposed action) approved
TicketPilot_3_Module_Component_Diagram.drawio	The backend's modules (AgentOrchestrator, tools, ApprovalEngine, services) and how they call each other
TicketPilot_4_Database_ER_Diagram.drawio	The eight tables and their relationships

Hosting, Deployment & AI/LLM Configuration

Backend: FastAPI app containerized with Docker, deployed to a low-cost host (Render/Railway/a small VPS). MySQL for relational data; Qdrant Cloud's free tier (or a self-hosted Qdrant container) for the knowledge-base embeddings.

Frontend: React + Redux app, built and deployed as a static site (Vercel/Netlify), calling the backend over REST.

LLM configuration: the LLM provider, model name, and API key are read through utils/ConfigReader.py from config.ini, never hardcoded — so the model can be swapped without a code change. Tool-calling uses the provider's native function-calling API with a fixed schema for each tool (check_order_status, check_account_history, search_knowledge_base, propose_action, escalate_ticket).

Logging: utils/Logger.py configures Python's standard logging module once (console handler, optionally a rotating file handler) and every module calls get_logger(**name**) for consistent, leveled logs.

Assumptions and Out of Scope

Assumptions:

- Order and account data is seeded/simulated for QuickCrave rather than connected to a real food-delivery backend.
- A single LLM provider's function-calling API is used; multi-provider fallback is not part of the MVP.
- Only one approval tier exists (a support manager); multi-level approval chains are not included.

Out of scope for the MVP (possible later additions):

- Real payment-gateway integration for issuing refunds — the MVP simulates the refund execution.
- Multi-language ticket support.
- A full analytics dashboard (auto-resolved vs. escalated vs. pending-approval rates) — planned as a post-MVP addition.